

Music Intelligence Hub

Real-time Stream Processing & AI Personalization Pipeline

Kafka • Spark • ClickHouse • Airflow

Problem Statement

The Latency Problem : Legacy batch systems process data in intervals, resulting in stale recommendations. When a user changes their mood, the platform fails to adapt instantly, leading to Irrelevant Discovery and user churn.

The Real-time Solution : An event-driven Lakehouse architecture that ingests millions of events per second. By using Kafka & Spark, we deliver AI-powered song suggestions in under 100ms, matching the listener's exact current context.

Lakehouse Architecture Flow

- i. **Ingestion:** Kafka buffers music events via simulators.
- ii. **Transformation:** Spark Streaming processes micro-batches.
- iii. **Storage:** MinIO & Delta Lake provide ACID reliability.
- iv. **Serving:** ClickHouse Speed Layer powers instant OLAP.
- v. **AI Engine:** Airflow orchestrates periodic recommendation generation.

Ingestion Layer: Dynamic Stream Simulation

Event Simulation: We use an automated process to generate realistic user music interactions like "PLAY", "SKIP", and "LIKE".

Data Flow: The system pulls user and song profiles from our storage and streams these interactions in real-time into the pipeline.

Reliability: The process is managed by Airflow, which ensures the simulation runs consistently and automatically restarts if any interruptions occur.

Compute: Spark Streaming

- Bronze Layer:** Real-time ingestion from Kafka with automated schema validation and quality filtering.
- Silver Layer:** Distributed enrichment by joining event streams with user and song metadata from Delta Lake.
- Gold Layer:** Final transformation and low-latency sink into ClickHouse for instant analytics.

Reliability: MinIO & Delta Lake

- ACID-Compliant Storage:** We use Delta Lake on top of MinIO S3 to ensure full transactional integrity, guaranteeing that data is never lost or corrupted.
- Time Travel:** The system supports querying historical versions of our data, allowing for easy auditing and A/B testing.
- Schema Enforcement:** We automatically prevent data corruption at the storage level, ensuring only clean and validated data enters our pipeline.

Serving: ClickHouse OLAP

- i. **High-Performance Analytics:** Utilizes ClickHouse as the speed layer to provide sub-second query responses for real-time dashboards.
- ii. **Real-time Sink:** Spark Structured Streaming pushes processed Gold-layer data directly to ClickHouse via optimized foreachBatch logic.
- iii. **Scalable OLAP:** Designed for massive aggregation queries, enabling instant visualization of live music trends, user behavior, and recommendation metrics.

AI Recommendation Logic

- **Logic Orchestration:** Uses Airflow to schedule an asynchronous Python-based engine that processes user interactions.
- **Intelligence Pipeline:**
 - Fetches real-time user profiles from ClickHouse.
 - Generates Top-N song recommendations per country using historical play trends.
- **Continuous Updates:** Injects personalized recommendations back into ClickHouse (user_recommendations table) using JSONEachRow for instant access by the frontend.

Tech Stack Justification

Feature	Apache Kafka	Traditional Brokers (e.g., RabbitMQ)
Data Retention	Persists messages for a set time (Replayable).	Deletes messages once consumed (Transient).
Scale	Built for massive, high-velocity data.	Optimized for complex routing and task queues.
Performance	High throughput via sequential disk I/O.	Fast for small messages, slower at scale.
Use Case	Event streaming and Data Pipelines.	Simple messaging and task distribution.

Feature	MinIO (Object Storage)	Hadoop (HDFS)	PostgreSQL (RDBMS)
Data Type	Unstructured (Data Lake)	Large Files (Big Data)	Structured (Transactional)
Scalability	High (Cloud-Native/S3)	Complex (Requires Clusters)	Low (Vertical Scaling)
Maintenance	Minimal (Containerized)	Very High (Monolithic)	High (Schema-bound)
Best Use Case	Modern Data Lakehouse	Legacy Big Data Systems	Transactional Apps

Tech Stack Justification

Feature	Apache Spark	Apache Flink	Pandas
Processing Type	Micro-batch (Unified)	Native Stream (Event-by-event)	In-memory (Single-node)
Scalability	High (Distributed)	High (Distributed)	Low (Limited to single machine)
Performance	Excellent for Large Datasets	Best for Ultra-low Latency	High for Small Data, Slow for Big
Use Case	Data Pipelines & Lakehouse	Real-time Complex Processing	Data Exploration & Small Analysis

Feature	ClickHouse (Columnar)	PostgreSQL (RDBMS)	MongoDB (NoSQL)
Data Storage	Column-oriented	Row-oriented	Document-based
Analytical Speed	Extremely Fast (Aggregations)	Slow for massive analytics	Moderate
Scalability	High (Sharding/Distributed)	Limited (Vertical)	High (Horizontal)
Use Case	Real-time Big Data Analytics	Transactional (ACID) apps	Flexible schema data

Tech Stack Justification

Feature	Apache Airflow	Cron Jobs	Prefect / Dagster
Monitoring	Built-in UI (Graphs/Logs)	None	Advanced UI
Dependency	Directed Acyclic Graphs (DAGs)	None (Sequential)	Complex Orchestration
Retry Logic	Native/Built-in	Manual Scripts	Modern/Built-in
Scalability	High (Distributed Workers)	Low (Single-server)	High

Challenges & Solutions



1. **Spotify API**
2. **Jars**
3. **Clickhouse conn**
4. **Spark Health Monitor**



Thank you.

Mohammed Luqman